DeadCell_Refactor

C++：面向对象：将数据和操作数据的方法封装到一起

C++/C#

C#

C++：面向对象：将数据和操作数据的方法封装到一起

构造函数：无返回值，用于初始化，默认不带参，可以传参用于初始赋值。（初始化顺序取决于声明顺序）

非成员函数，子类只能通过super（）调用父类的构造函数，无法override

~虚析构函数：使用基类指针指向派生类对象时，基类不为虚函数那么通过基类指针delete对象时只会调用基类的，容易造成派生类内部动态分配的资源泄露。

纯虚函数：包含至少一个纯虚函数的类成为抽象类，不能实例化（不能做形参/返回值），不能直接创建对象只能创建它的具体子类对象，只能作为基类被继承，用于定义接口规范，约定函数的入参，返回值和功能，不写实现。实现多态和接口隔离。函数体?

普通成员方法/成员函数：隶属于类，可访问私有成员。自带this指针（隐藏的指针常量函数，值为对象的内存地址，栈上）指向调用它的对象本身，可以访问所有成员变量。静态成员方法无this，只能访问静态成员变量和其他静态方法。

多态：“多态的底层是通过虚函数表和虚指针实现的。有虚函数的类会在编译期生成一张虚函数表vtable（只读数据段/常量区），表里存储所有虚函数的入口地址（函数指针）；每个对象的头部会有一个虚指针vptr，指向当前类的虚表。当用基类指针调用虚函数时，会先通过对象的虚指针找到对应虚表，再查表调用对应函数，最终调用哪个版本由对象的实际类型决定，以此实现运行时多态。

对比维度	成员变量（内置类型，如int/float）	局部变量（内置类型，如int/float）
默认值	分两种情况：1. 全局 / 静态对象的成员 → 自动零初始化（值为 0、指针为 nullptr）2. 栈 / 堆上的普通对象的成员 → **未初始化，是随机垃圾值**	无默认值，未初始化就是**随机脏数据**，直接使用会触发未定义行为，必须手动赋值

存储位置	跟随对象所在的内存区域：对象在栈 → 成员在栈对象在堆（new 创建）→ 成员在堆静态成员固定在全局数据区	统一存储在**栈区**
生命周期	和所属对象同生共死，对象销毁就释放；静态成员全程存在	仅在所属代码块执行期间有效，代码块结束（函数返回、循环结束）就自动释放
作用域	类内部所有方法都能访问；对外访问权限由 public/protected/private 控制	仅在定义它的代码块内可见，出了范围就无法访问

static:静态成员属于类本身，在类外赋值。

优先级：

深浅拷贝

序列化

左右值：int a=10,a左值：有名字，能取地址 10：临时对象

函数	写法	触发场景
默认构造	`Test()`	定义对象不传参时

拷贝构造	`Test(const Test&)`	用**左值**创建新对象时，如 `Test c = b;`
移动构造	`Test(Test&&)`	用**右值**创建新对象时，如 `Test c = move(b);`
拷贝赋值	`Test& operator=(const Test&)`	两个对象都已存在，左值之间赋值，如 `a = b;`
移动赋值	`Test& operator=(Test&&)`	两个对象都已存在，用右值赋值，如 `a = move(b);`

C++/C#

P0

必补基础 C++ 内存分区：栈、堆、全局 / 静态存储区、常量区、代码区，各自的特点、生命周期，对应值类型 / 引用类型的底层逻辑 空类大小、含虚函数的类大小、多继承下的虚函数表布局（校招常考大小计算） 构造函数不能是虚函数的原因（和析构函数虚函数配套追问，10 问 9 跟） struct 和 class 的默认访问权限区别

【构造函数：对象出生时初始化，可重载；析构函数：不可重载，无参数。；虚函数：（vptr对象+vtable类）构造函数负责对象创建时的初始化和资源申请，析构函数负责对象销毁时的资源释放，分别对应对象生命周期的起点和终点。

虚函数的底层靠虚指针和虚表实现：每个有虚函数的类，编译期就会生成一张虚表，存在代码区的只读数据段，里面存着所有虚函数的地址；每个对象实例里有一个虚指针，在构造函数运行时初始化，指向对应类的虚表。调用虚函数时，先通过虚指针找虚表，再找到对应函数地址执行，以此实现运行时多态。

构造函数不能是虚函数，一是因为构造时虚指针还没初始化，找不到虚表；二是构造对象时类型是确定的，不需要多态。而基类的析构函数建议设为虚函数，是为了在用基类指针销毁派生类对象时，能正确调用派生类的析构函数，避免内存泄漏】

STL 必补高频坑点 vector 迭代器失效的场景（扩容、erase/insert）及解决方法，游戏开发高频踩坑点 deque 的底层结构和优缺点（和 vector 对比常考）

优先级下调（不用死磕，了解即可） 移动语义、std::move、右值引用：中小厂校招很少深挖，知道概念、能说清避免深拷贝就行，不用抠底层实现 Lambda 表达式捕获：知道值捕获 / 引用捕获

的区别即可，不会考太细

C# 与 Unity 基础:Unity 生命周期（面试开篇第一题概率 90%） 完整执行顺序：Awake → OnEnable → Start → FixedUpdate → Update → LateUpdate → OnDisable → OnDestroy 每一个方法的执行时机、典型使用场景；Update 和 FixedUpdate 和 LateUpdate 的本质区别

Unity 脚本生命周期主要分四个阶段：初始化、帧循环、渲染、销毁。 初始化阶段按 Awake→OnEnable→Start 顺序执行：Awake 是组件实例化完成就调用，和启用无关，用来做自身初始化；OnEnable 每次启用都会触发，适合注册事件；Start 在第一帧 Update 前执行，用来做依赖外部的初始化。 帧循环分两套时钟：FixedUpdate 是固定物理步长，和帧率无关，用来处理物理逻辑；Update 每帧调用，处理主游戏逻辑；LateUpdate 在所有 Update 之后执行，适合相机跟随这类后处理逻辑。 最后销毁阶段先触发 OnDisable，再触发 OnDestroy，用来做事件注销和资源释放。

- **协程（Unity 特有，必问）**
- 协程本质：不是多线程，是迭代器（IEnumerator），在 `Update` 之后逐帧执行
- 常见 `yield return` 类型的执行时机： `null` 、 `WaitForSeconds` 、 `WaitForEndOfFrame` 、 `WaitForFixedUpdate`
- 协程的终止方式，协程的常见坑（对象销毁后协程还在跑）

协程本质是用户态的协作式调度，不是操作系统级别的多线程。它的底层是 C# 的迭代器状态机，编译器会把带 yield return 的代码编译成一个状态机类，保存执行位置和局部变量，通过 MoveNext 方法逐段执行。 程适合做延时、分帧加载、异步等待这类 IO 型 / 等待型逻辑；CPU 密集型计算必须用真正的多线程（Task、线程池） ：协程的隐患主要集中在生命周期、内存和逻辑三个方面。

生命周期上，脚本禁用、物体失活都不会自动停止协程，只有销毁对象才会，很容易出现对象已经没了，协程还在跑导致空引用；而且 StopCoroutine 必须保存 Coroutine 引用才能可靠停止，直

接传迭代器是停不掉的。

内存上，挂起不结束的协程会一直持有外部引用，导致对象无法被 GC 回收，造成内存泄漏；另外高频 new WaitForSeconds 这类等待指令会产生持续 GC，一般会全局缓存复用。

逻辑上，循环里开协程容易有闭包变量捕获的问题，try-catch 也不能跨 yield 捕获异常，嵌套协程停止时也不会联动，这些都是实际开发里容易踩的坑。

最后协程是单线程分帧执行，解决不了 CPU 密集型卡顿，海量实例也不适合每个都开协程，会有额外的调度开销。

Unity 把这些迭代器注册到主循环里，在不同的生命周期时机调用 MoveNext，不同的 yield 指令对应不同的恢复时机，比如 null 是下一帧 Update 之后，WaitForFixedUpdate 是物理帧之后。整个过程完全运行在主线程上，操作系统感知不到，也没有线程切换的开销。

它和多线程的核心区别是：线程是内核抢占式调度，可以多核并行；协程是引擎协作式调度，只能在主线程里分帧执行，是并发不是并行。所以协程适合做延时、分帧加载、异步资源等待这类逻辑，不适合做 CPU 密集型计算。

- **资源管理（游戏开发核心，必问）**
- Resources / AssetBundle / Addressables 三种加载方式的区别、优缺点、适用场景
- AB 包卸载：`Unload(false)` 和 `Unload(true)` 的本质区别，内存泄漏风险
- 资源依赖关系怎么处理，AB 包打包的常见坑

Resources 适合小体量原型，不能热更，商业项目用得少；原生 AssetBundle 灵活可控，支持热更，但依赖和内存都要手动管理，容易踩冗余和泄漏的坑；Addressables 是官方现在主推的方

案，基于 AB 封装，自动处理依赖和引用计数卸载，迭代效率高，也支持热更新。 AB 卸载分两种：Unload (false) 只卸 AB 包本身，已加载的资源保留，但容易出现脱离管理的内存泄漏；Unload (true) 会把 AB 和所有对应资源一起销毁，清理彻底但容易误删在用资源。 依赖方面，原生 AB 要手动加载完整依赖链Manifest，Addressables 则是全自动管理，不用开发者关心底层细节。

- **UGUI 优化细节**
- DrawCall 合批的两个核心条件：相同材质 + 相同图集 + 渲染顺序连续
- Canvas 动静分离的原理：为什么静态 UI 不要动，动态 UI 要拆到单独 Canvas
- 网格重建（Rebuild）的触发条件，怎么减少重建

打图集是 UI 合批的基础，它把多张零散图片合并成一张大贴图，让多个 UI 元素共用同一个材质，满足了合批的材质前提。但合批不是打了图集就自动完成，引擎只会在同一个 Canvas 内，把材质相同、排序一致、渲染顺序连续的元素合并成一批。

实际开发中，除了打图集，还要注意避免图文穿插、控制 Order in Layer 的使用、减少不必要的

Mask，才能最大化合批效果，降低 DrawCall。

- **Unity GC 全链路**
- C# GC 的触发时机
- Unity 中常见的 GC 产生点：字符串拼接、foreach 循环、数组分配、GetComponent 频繁调用、Unity API 返回值装箱
- 减少 GC 的常用手段：对象池、字符串复用、避免装箱拆箱

【最高优先级：Update/FixedUpdate 等高频逻辑零分配，这是卡顿的最主要来源。 次高优先级：高频创建销毁的对象做池化 / 缓存复用，比如子弹、特效、临时集合。 细节优化：避免装箱、优化字符串、避开 API 隐式分配，积少成多。 装箱只是 GC 来源中的一个细节点，不是核心大头；真正占比高的是频繁 new 类对象、字符串操作、Unity API 隐式分配这些。面试标准口述版 Unity 的 GC 基于托管堆的标记 – 清除算法，回收时会触发 Stop-The-World 暂停主线程，造

成画面卡顿。优化的核心思路是从源头减少托管堆分配，而不是依赖事后回收。 具体手段包括：高频创建销毁的对象用对象池复用，临时对象做静态缓存，集合初始化指定容量；字符串高频拼接用 StringBuilder，减少临时字符串生成；通过泛型、重写方法等避免装箱拆箱；避开 Unity API 中返回数组的隐式分配，改用带结果参数的重载；高频逻辑中避免 new 对象和闭包；最后在过场、加载等安全点主动触发 GC，避开战斗等关键帧。】

- 网络基础：补全 TCP 必问链路 TCP 三次握手、四次挥手的过程，为什么是三次不是两次？为什么四次挥手？ TIME_WAIT 状态的作用 粘包拆包的产生原因和解决方法（游戏开发常用） 状态同步 vs 帧同步：你有项目理解，能说清区别、优缺点、适用品类就行，不用抠底层实现

P1

设计模式：全部结合你的项目讲，不要空背定义

- **单例模式**：饿汉、懒汉、线程安全的写法；Unity 里 Mono 单例和普通单例的区别；单例的优缺点
- 你项目里对应的模式：
  - 观察者模式 → EventCenter 事件总线
  - 组合模式 → 行为树节点结构
  - 享元模式 → 对象池思想
  - 工厂模式 → 武器 / 敌人创建 面试时直接说「我在 XX 项目里用 XX 模式解决了 XX 问题」，比纯背定义强 10 倍。

2. 操作系统：抓核心考点

- 进程和线程的区别、进程间通信的 5 种方式
- 线程同步的方式：互斥锁、信号量、条件变量
- 死锁四条件 + 预防 / 避免方法

3. C# 进阶：

- 反射与特性：`[SerializeField]` 的原理，反射的优缺点和适用场景
- 多线程：Thread、Task、线程池的区别；Unity 中为什么不能在子线程改 UI
- 闭包的原理和常见坑

4. Unity 进阶

- Shader 基础：渲染管线的大致流程（应用阶段→几何阶段→光栅化阶段）；漫反射、高光反射的基础光照模型
- 动画系统：Animator 和 Animation 的区别；根运动（Root Motion）的原理和用法

p2:

xLua 热更新 xLua 的核心原理：C# 端通过 IL 注入，Lua 端通过委托 / 接口替换业务逻辑，实现运行时热更 C# 调用 Lua、Lua 调用 C# 的方式，交互的性能开销点 你项目里用 Lua 写状态机的优势：不用改代码、不用重打包，直接热更逻辑 性能优化工具 Unity Profiler 的常用模块：CPU、内存、渲染；怎么定位 GC 峰值、DrawCall 过高、内存泄漏 战斗系统专项（你的独家优势） 常见伤害判定方式：重叠检测（Overlap）、射线检测、碰撞体触发，各自的适用场景 技能 Timeline 的执行流程：怎么对齐伤害帧、特效帧、音效帧 这部分别人没实习经验讲不出来，你能说清楚就是碾压级加分。

C#

📎HFSM_知识整理.md

我本身职业规划就是客户端开发，这份策划实习是我主动选择的过渡 —— 我想先完整了解游戏项目的开发全流程，再切入开发岗，避免只会写代码、不懂业务逻辑。

实习期间我主要负责战斗系统的规则设计、逻辑校验和跨部门联调，虽然没有直接写项目的业务代码，但全程深度参与了需求评审、接口对齐、bug 定位的完整环节，对客户端和服务器的协作模式、数据驱动的开发流程都有很直观的认知。同时我也一直在参考公司的项目架构，用业余时间优化自己的 Demo，比如用 Lua 重构了分层状态机，把学到的设计思路落地。

之所以不继续做策划，是因为我更享受用代码解决问题的过程，也希望在技术方向上深耕，策划的经历对我来说是加分项，而不是转行的退路。

策划转开发的短板我很清楚，就是商业项目的代码开发经验不足，所以我这段时间一直在集中补客户端的核心基础，刷算法、复盘 Unity 底层原理，同时做开发练习补实操经验。

如果能入职，前三个月我首先会快速吃透负责模块的代码架构和开发规范，熟悉协作流程；其次凭借我之前的策划经验，能快速理解需求背景，提前预判边界问题，争取尽快能独立负责简单的功能开发和问题修复。

和纯科班的候选人比，我不算传统意义上的野路子，我的优势在于懂完整的项目链路，开发时不会只局限于代码实现本身，能更好地对齐需求、减少返工，上手业务的速度会更快。

HFSM：以状态为核心，分层嵌套，状态的转移是固定的，确定性的。优势：逻辑直观，性能比较好，调试起来比较简单。缺点：难以处理复杂的决策逻辑，拓展性差，会有冗余

行为树：以决策节点为核心，通过顺序，选择，并行等节点组合实现复杂逻辑。优势：灵活，拓展性强，适合配置复杂的ai；缺点是性能比HFSM差，调试成本高。

重构前用的是普通单层级 FSM，状态少的时候确实能用，但有两个很明显的问题：一是所有状态逻辑都堆在 PlayerController 里，代码涨到 400 多行，改一个状态很容易误碰其他逻辑；二是状态转换重复冗余，比如受击、死亡要打断所有动作，我得在行走、跳跃、攻击每个状态里都写一遍判断，新增状态还要补，很容易漏。

刚好我在公司接触到了 Lua+Unity 的热更架构，也看到了项目里分层状态机的设计，就想着用自己的项目落地一遍，既解决现有问题，也练一下 Lua 开发。最终我把状态机全部迁到了 Lua 层，C# 只保留状

态机驱动的核心接口，状态逻辑全用 Lua 实现，支持热更。

设计上我用了分层状态机，一共分三层：最上层是异常层（受击、死亡），优先级最高；中间是主动行为层（攻击、冲刺）；最底层是基础移动层（地面、空中）。分层的核心好处是复用转换逻辑：高优先级的异常状态可以直接打断所有低优先级状态，不用在每个子状态里重复写判断；基础层的移动切换也不用侵入上层行为逻辑，状态数量越多，收益越明显。

状态创建我用了工厂模式统一管理，对外只暴露状态类型枚举，内部负责创建对应状态对象，上层不用关心状态的具体实现。状态切换的时候，统一走基类的 Enter/Exit 生命周期，在 Exit 里做事件解绑、数据清理，Enter 里做初始化，避免状态残留。

优先级处理我用的是前置判断机制：切换状态前先比较目标状态和当前状态的优先级，只有目标优先级≥当前才允许切换，同优先级状态做互斥判断，避免循环切换的死锁问题。

重构之后的收益很直观：PlayerController 精简到了 200 行左右，只负责状态机调度和核心接口，职责更清晰；新增一个状态只要新建 Lua 文件、注册到工厂里就行，不用改动原有状态代码，扩展性好了很多；另外状态逻辑全在 Lua 层，后续逻辑出问题可以直接热更修复，不用走客户端整包更新。

行为树我也在敌人 AI 里用过，两者的适用场景不一样：HFSM 适合状态层级清晰、优先级明确的场景，比如玩家控制器，逻辑确定、性能好、调试直观；行为树更适合复杂的决策类逻辑，比如怪物 AI，灵活度高、方便配置复杂分支。玩家控制器状态相对固定，用 HFSM 性价比更高。

我这个项目的核心是把 C# 单层 FSM 重构成了 xLua 驱动的分层状态机，主要解决两个问题：一是原来的状态逻辑都堆在控制器里，代码冗余、维护成本高；二是 C# 写的逻辑没法热更，出问题必须整包更新。

架构上我分了三层：异常层优先级最高，独占执行，用来处理受击、死亡这种强制打断的逻辑；基础层管移动、跳跃这些底层状态；主动层管攻击、冲刺这些玩家行为，支持并行和互斥两种模式。分层之后，像受击打断这种通用逻辑只需要写一次，不用在每个状态里重复加判断，代码冗余少了很多，新增状态也只需要加单独的 Lua 文件。

实现上我用工厂模式统一创建状态对象，初始化就全部缓存复用，避免切换时产生 GC。Lua 和 C# 之间我做了一层接口封装，Lua 只做决策，不直接碰底层组件，保证了两边的解耦。后续状态逻辑出问题，可以直接热更对应的 Lua 文件，不用动 C# 代码。

GC 优化我主要做了两点： 第一，状态对象采用单例缓存复用，初始化时一次性创建所有状态实例并注册到状态机，切换状态仅修改当前指向，不产生任何新对象，把状态切换的 GC 开销降到了 0。 第二，针对 Lua 和 C# 的高频交互做了合并：把一帧内多次调用的移动参数打包传递，代替原来分好几次调用

Set 方法，减少跨语言调用的次数；同时把 Lua 侧的 Update 函数提前缓存成 C# 强类型委托，避免每帧反射调用的开销。

跨语言交互的性能坑我有过系统梳理，核心是反射、装箱和调用次数这三类问题。我自己的项目里，主要是尽量合并同帧的零散调用，减少跨域次数，同时缓存常用的 Lua 函数委托，避免每帧反射开销。写代码的时候也会刻意规避直接频繁访问 C# 字段、大量传字符串这类高成本写法。

我项目 UI 规模不大，所以优化上主要做基础规范和架构管理： 第一，严格遵守合批基础规则，同 Sort Layer、同材质的 UI 放在连续层级，不用透明元素打断合批；共用图标打成图集，减少材质数量，避免不必要的 DrawCall。 第二，用栈结构做了 UI 界面的生命周期管理，打开入栈、关闭出栈，复用界面对象，避免频繁创建销毁带来的 GC 和卡顿。

我之前做 FPS Demo 的时候专门处理过斜面移动的问题。核心思路是把玩家的移动方向向量，通过斜面法线投影到斜面的切平面上，得到真正沿斜面的移动方向，而不是直接沿世界坐标系水平移动。 这样既避免了移动向量怼进斜面导致的穿模、抖动，也能正确计算沿斜面的重力分量，保证贴地移动的手感。

我自己的项目里，性能优化主要集中在 Lua 层和 UI 基础层。 Lua 状态机这块，我做了状态对象的单例缓存复用，初始化一次性创建所有状态实例，切换状态只改指向不产生 GC；同时针对 Lua 和 C# 的跨语言调用，把同帧的多次操作打包合并，减少调用次数，也缓存了 Update 的委托避免反射开销。跨语言调用的几个核心坑，比如装箱拆箱、属性访问开销、字符串传递成本，我也都有了解，写代码的时候会刻意规避。 UI 方面因为项目规模不大，主要是做好基础规范，遵守 UGUI 的合批规则，共用资源打成图集，减少不必要的 DrawCall；同时用栈结构管理界面生命周期，复用对象避免频繁创建销毁。 3D 相关我之前做过 FPS 的 Demo，处理过斜面移动的问题，主要是通过向量投影把移动方向映射到斜面切平面上，解决穿模和贴地移动的手感问题